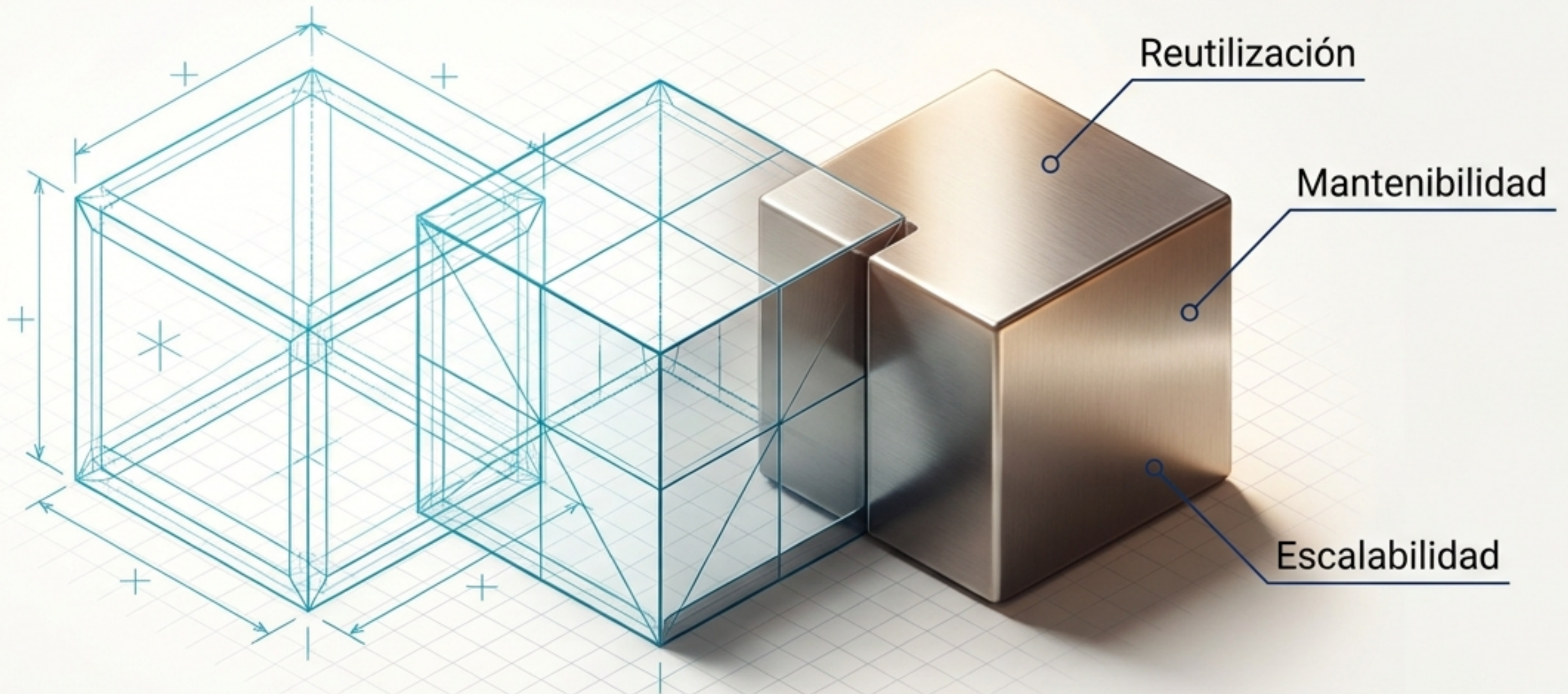
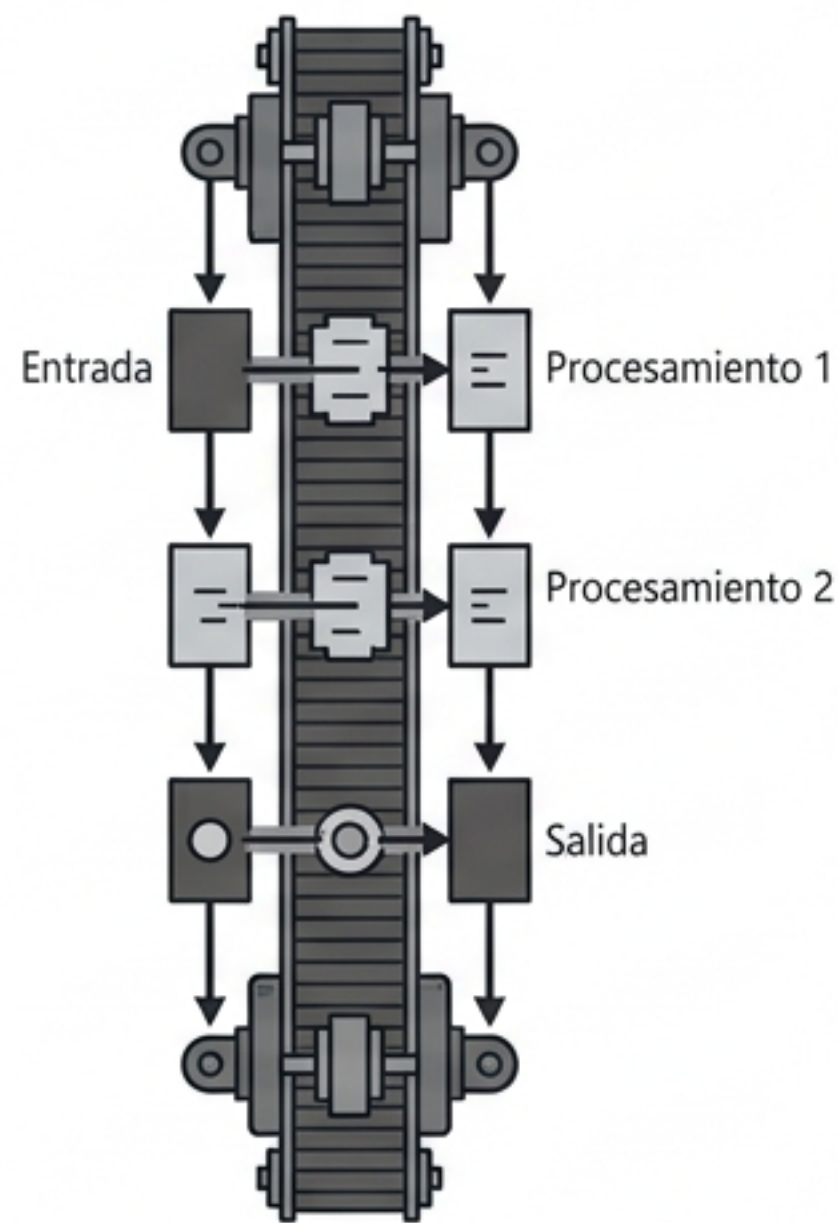


Construyendo Sistemas Complejos con C# y Programación Orientada a Objetos

Guía Visual Maestra de POO para Programación II

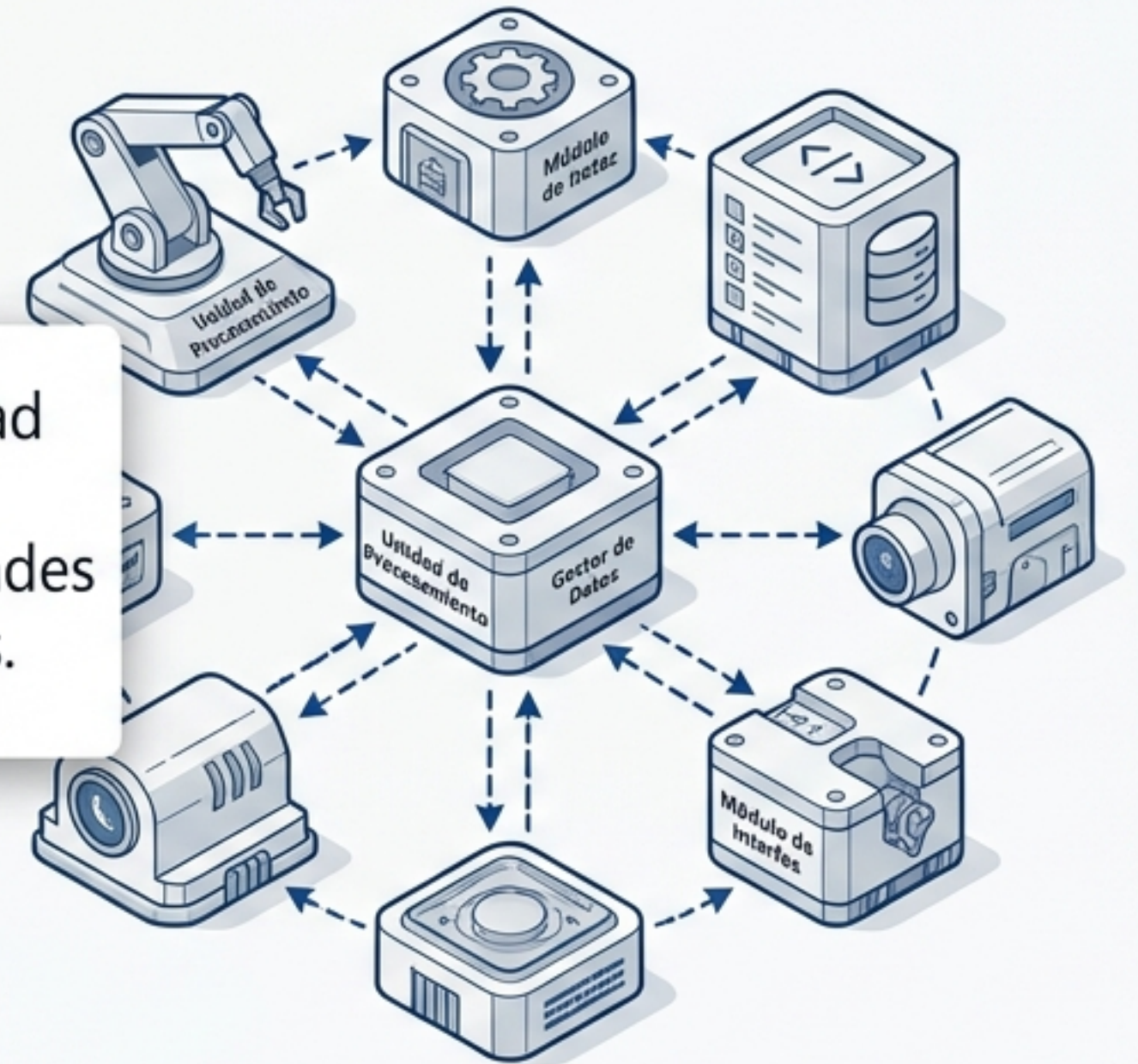


Programación Estructurada



La POO modela la realidad agrupando datos y comportamientos en unidades integradas y autónomas.

Programación Orientada a Objetos



Reutilización.

Herencia y composición reducen la redundancia del código.



Mantenibilidad.

Modularidad que facilita la depuración sistemática.

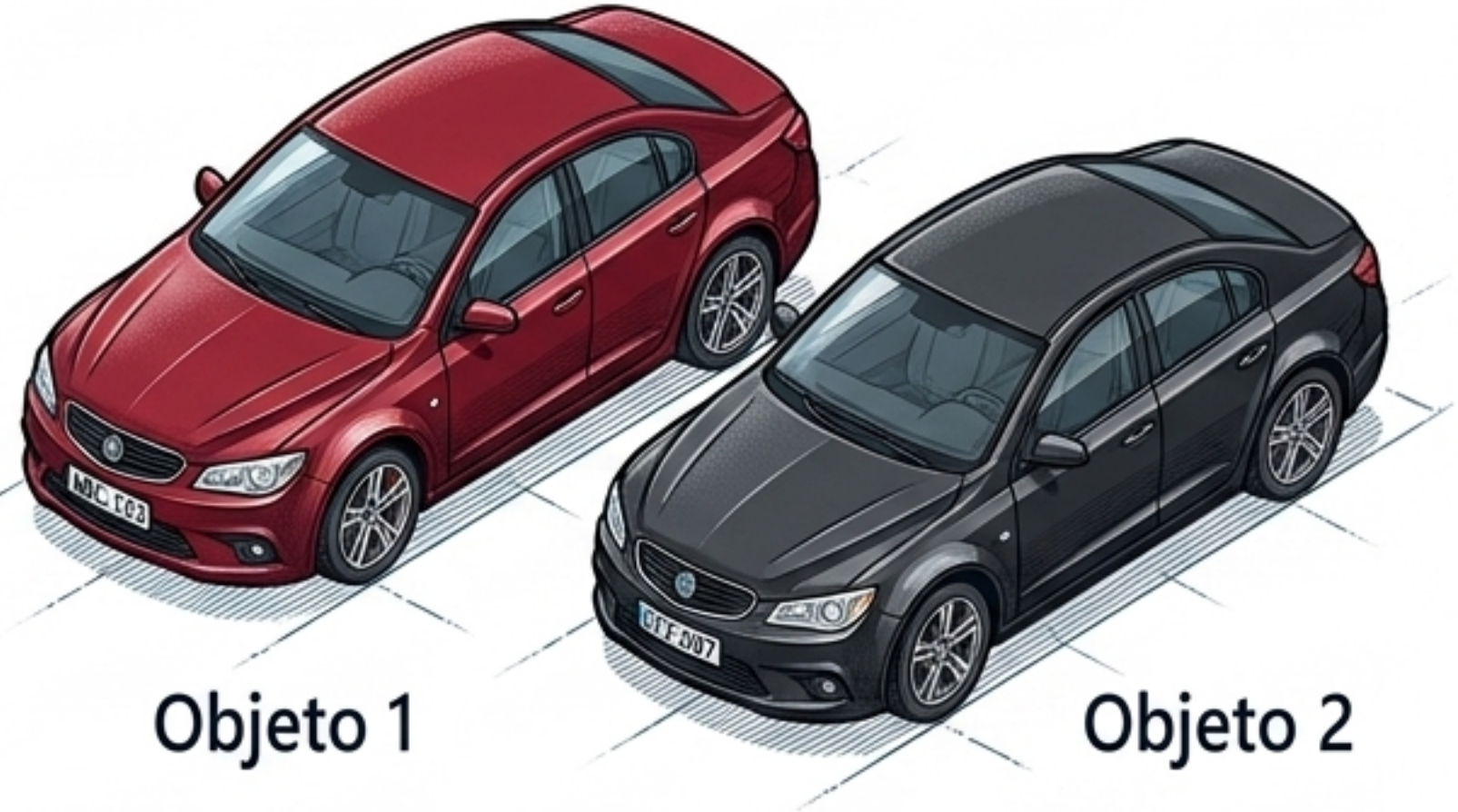
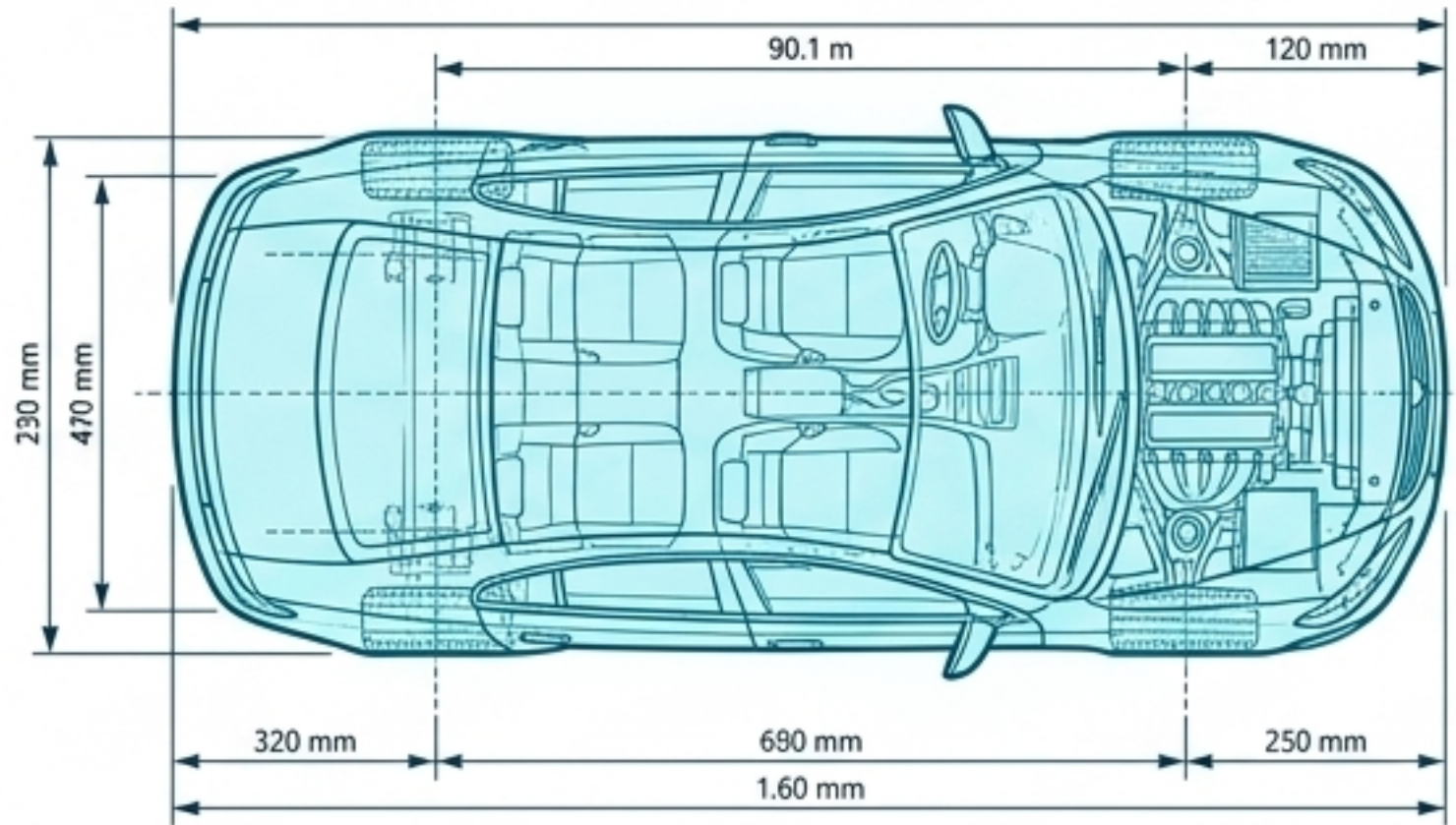


Escalabilidad.

División clara de responsabilidades para proyectos más grandes.

La Matriz Fundamental de la Arquitectura de Software

class Auto



La Clase (El Plano)	El Objeto (La Construcción)
Naturaleza: Plantilla genérica o molde estructural.	Naturaleza: Instancia física y concreta.
Memoria: No ocupa espacio de datos (solo define las reglas).	Memoria: Reside dinámicamente en la memoria RAM.
Identidad: Única en el código base.	Identidad: Múltiples instancias con estado propio (Ej. color, placa).

Anatomía Interna de una Clase en C#

```
public class Estudiante  
{  
    public string Nombre;  
    public double NotaExamen1;  
  
    public double CalcularPromedio()  
    {  
        return (NotaExamen1 + NotaExamen2) / 2.0;  
    }  
}
```

La Declaración.

Define el inicio del plano conceptual y la frontera lógica de la entidad.

Atributos (El Estado).

Variables que representan los datos que el objeto "sabe" sobre sí mismo.

Métodos (El Comportamiento).

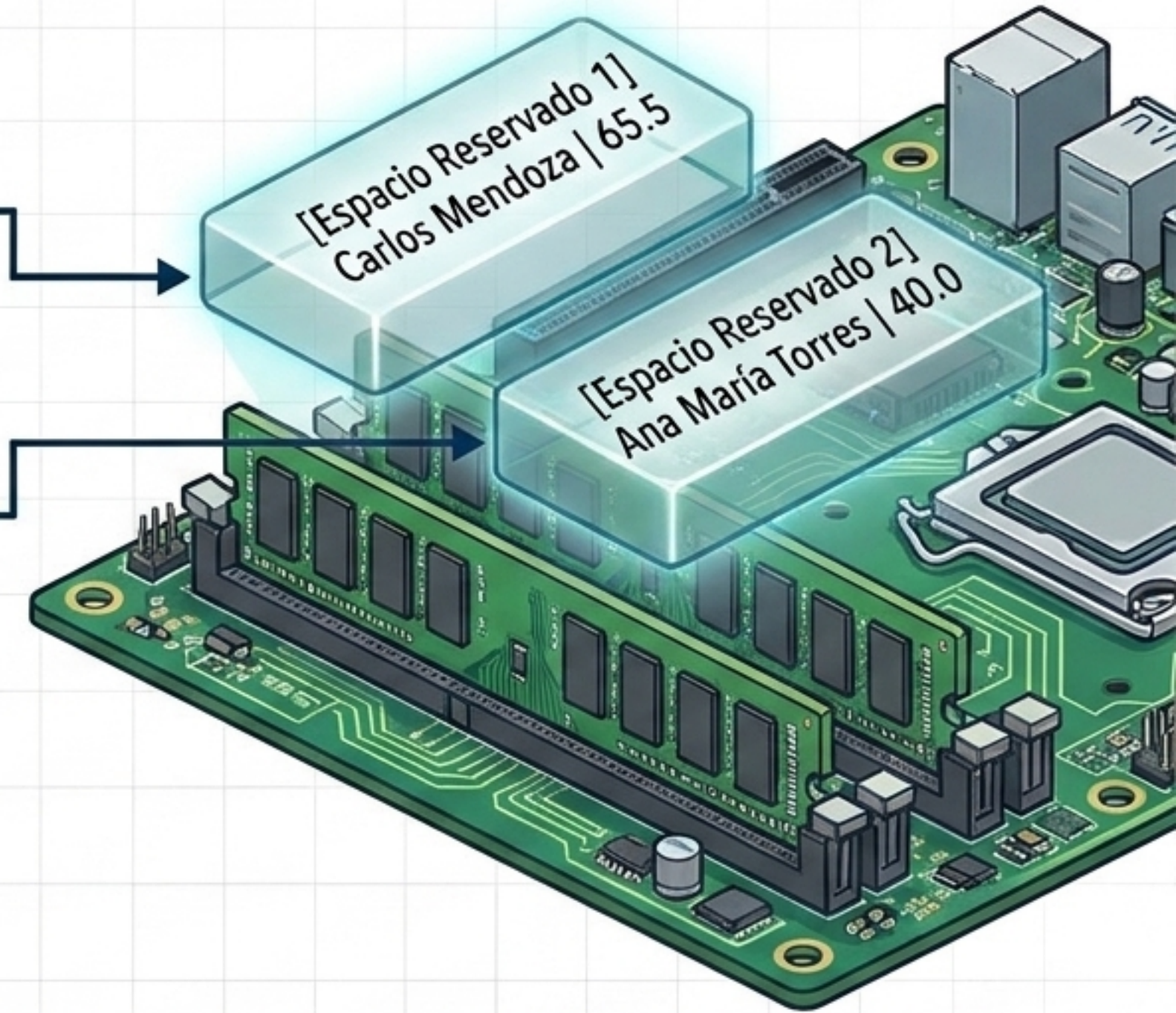
Operaciones o acciones lógicas que el objeto "puede ejecutar".

En C#, la clase agrupa variables y funciones bajo una misma frontera protectora.

El Proceso de Instanciación y Asignación de Memoria

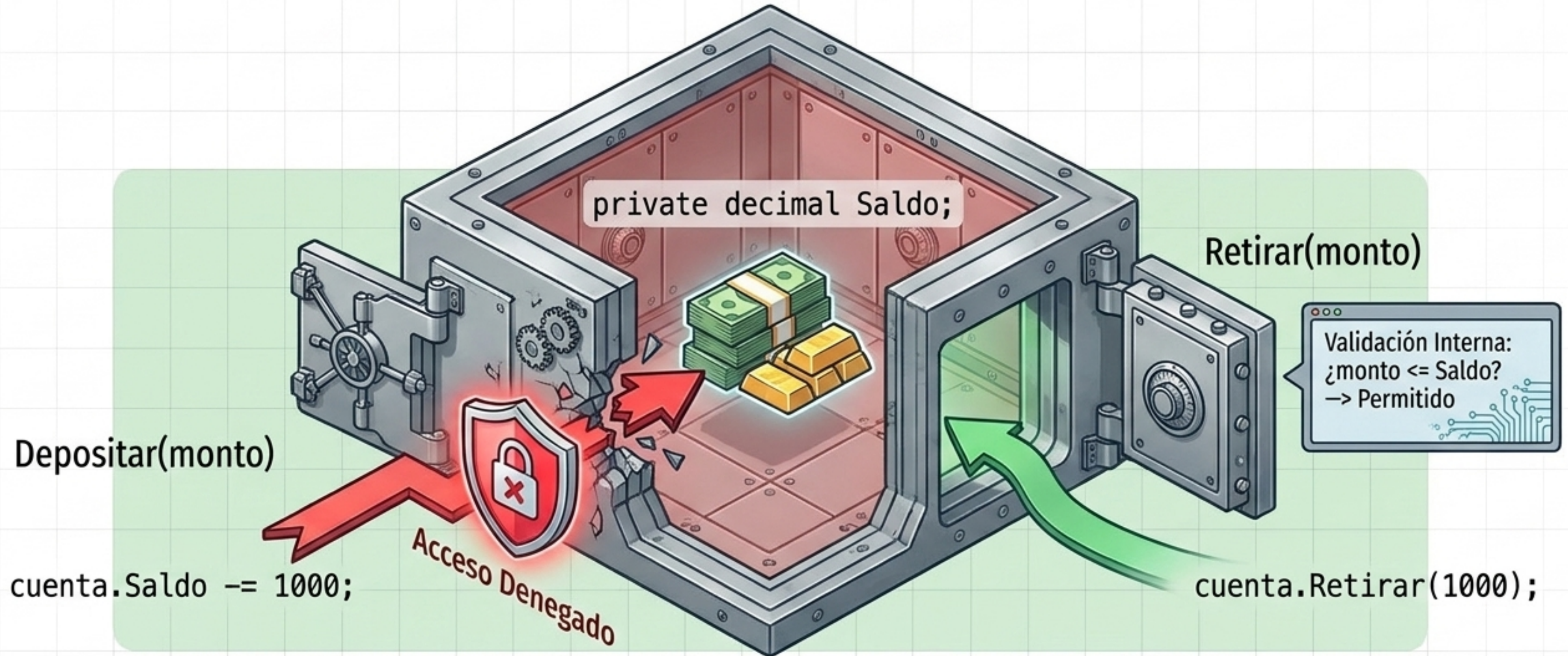
```
Estudiante est1 = new Estudiante();  
est1.Nombre = "Carlos Mendoza";  
est1.NotaExamen1 = 65.5;
```

```
Estudiante est2 = new Estudiante();  
est2.Nombre = "Ana María Torres";  
est2.NotaExamen1 = 40.0;
```



La palabra clave 'new' actúa como la fábrica. Toma el molde de la clase y reserva un bloque único en la memoria física para almacenar el estado independiente de cada nuevo objeto.

Ocultando la Complejidad y Protegiendo la Integridad



El objeto defiende sus propios datos. Nadie modifica el saldo directamente; todos los accesos externos deben pasar por las validaciones de seguridad de los métodos públicos.

Matriz de Diagnóstico: Modificadores de Acceso

Modificador	Misma Clase	Clase Derivada	Mismo Proyecto (Assembly)	Cualquier Lugar
public (Puerta Abierta) 	✓	✓	✓	✓
internal (Edificio) 	✓	✓	✓	✗
protected (Árbol) 	✓	✓	✗	✗
private (Candado Cerrado) 	✓	✗	✗	✗



Regla de Oro en C#: Si omites el modificador de acceso en una declaración, el compilador asume estrictamente que es 'private' por defecto.

Propiedades y la Capa de Validación Integrada

```
private decimal _precio;
```

```
public decimal Precio  
{
```

```
    get { return _precio; }
```

```
    set
```

```
{
```

```
        if (value >= 0)  
            _precio = value;
```

```
    }
```

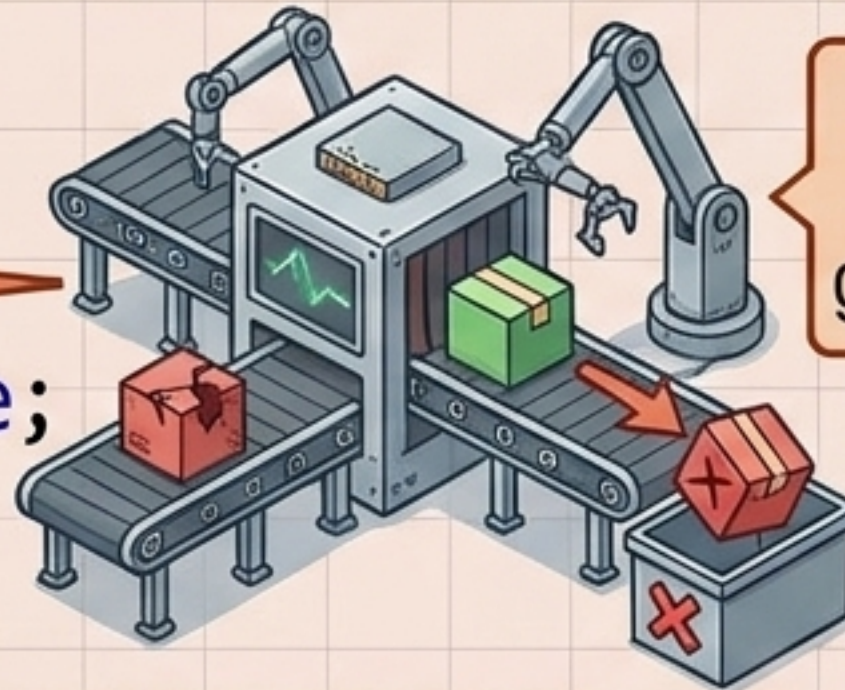
```
}
```



Materia Prima (Oculta)



**Ventana de Exhibición
(Solo Lectura)**



Inspector de Calidad: Rechaza precios negativos antes de guardarlos en la variable privada.

Atajo Moderno:
Propiedades Automáticas

```
public int Stock  
{ get; private set; }
```


El Ciclo de Vida del Objeto en Memoria



Nacimiento: El Constructor

Se invoca con 'new'. Tiene el mismo nombre de la clase y su única misión es inicializar el estado base del objeto en la RAM.

Desarrollo: Ejecución

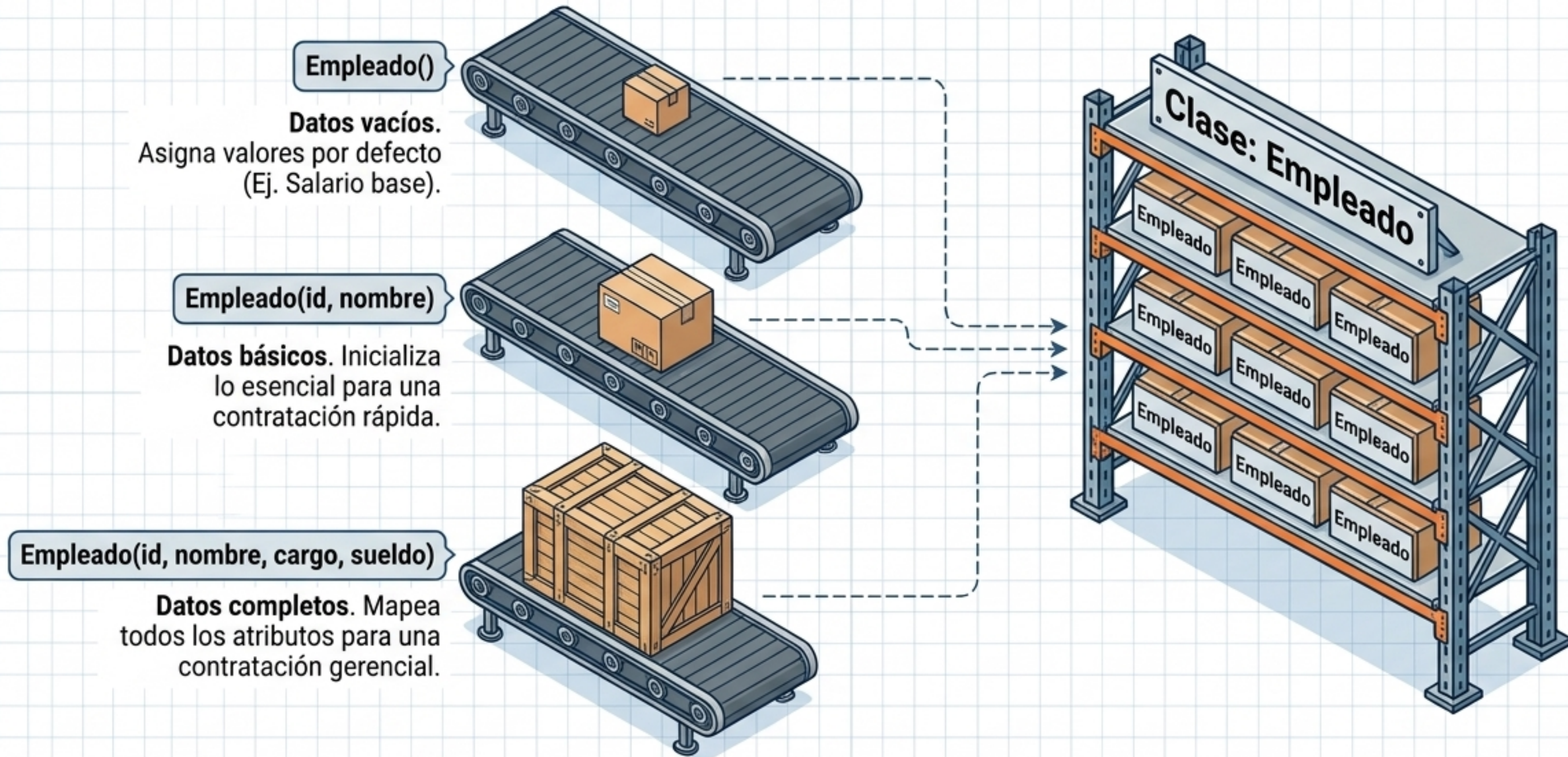
El objeto reside en memoria. El sistema invoca sus métodos, lee sus propiedades y altera su estado según la lógica de negocio.

Demolición: Destructor / GC

El Recolector de Basura (Garbage Collector) de .NET detecta que el objeto está huérfano y limpia la memoria automáticamente.

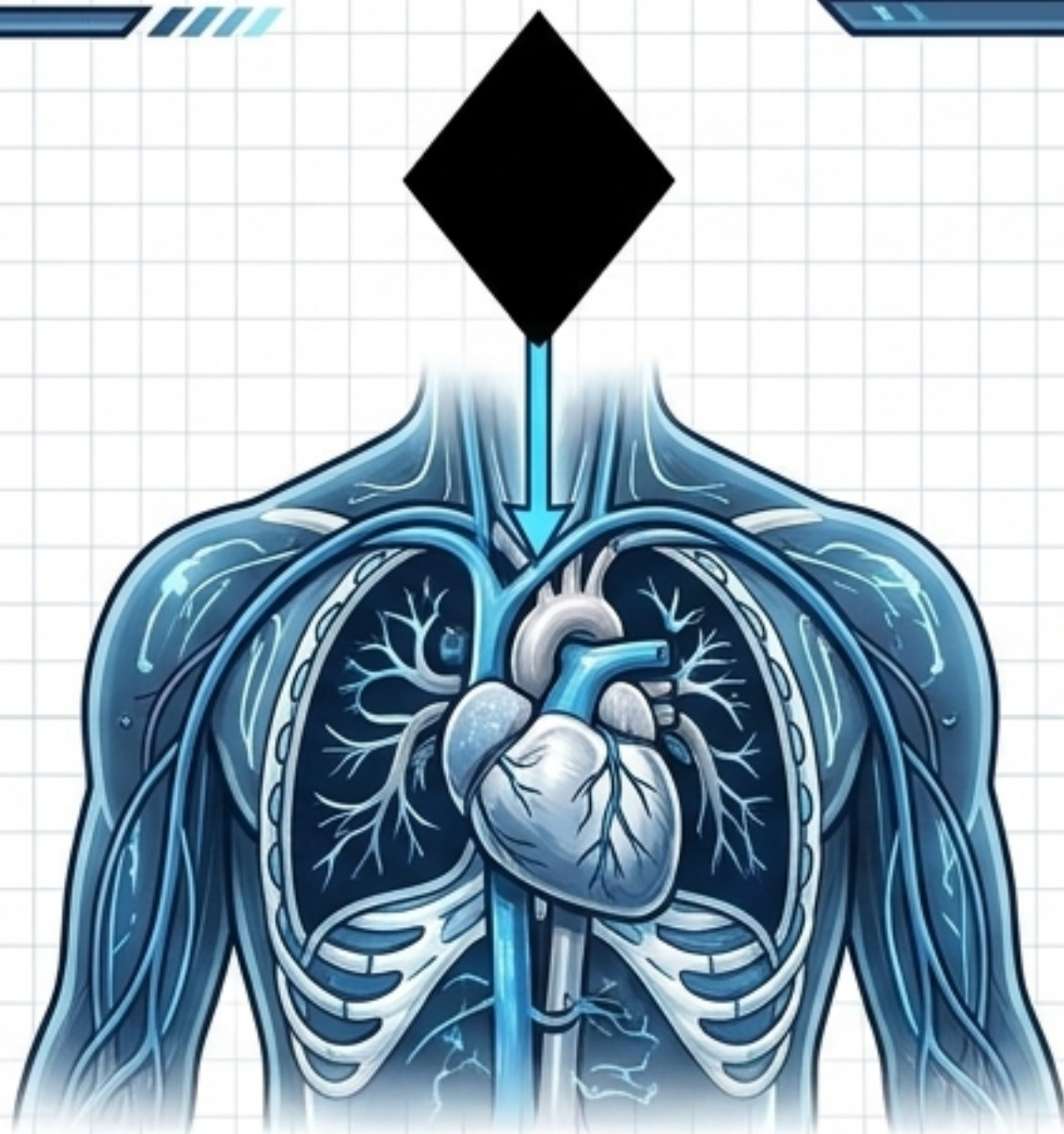
*Nota de Arquitectura: En C# moderno, la limpieza manual de recursos no administrados se gestiona preferiblemente con la interfaz `IDisposable` y bloques 'using'.

Sobrecarga de Constructores y Adaptabilidad



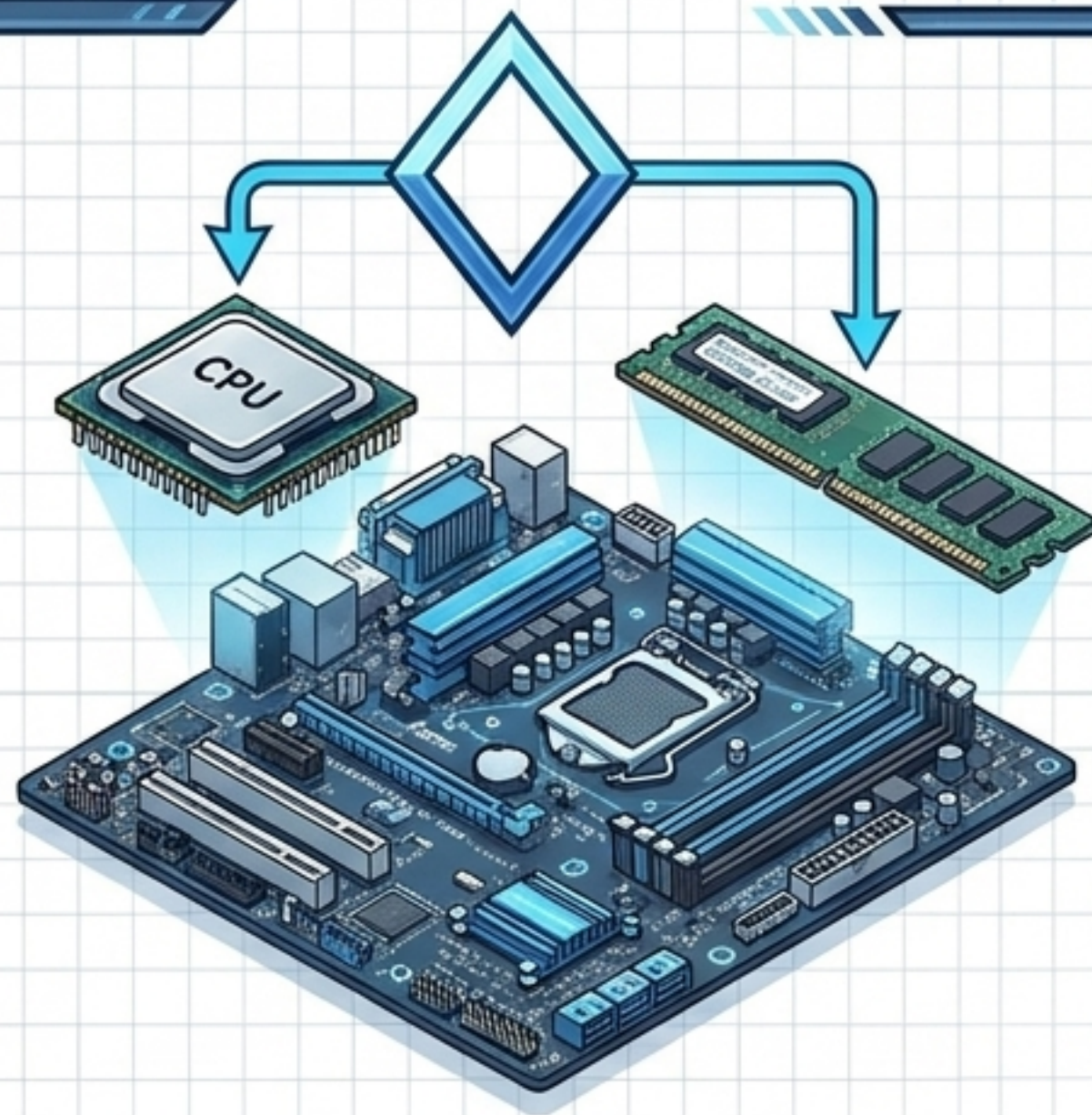
Regla Visual: Mismo nombre de método + Diferente firma de parámetros = Sobrecarga.
El objeto se adapta inteligentemente a los datos disponibles al nacer.

Ensamblando Sistemas Modulares (Composición vs. Agregación)



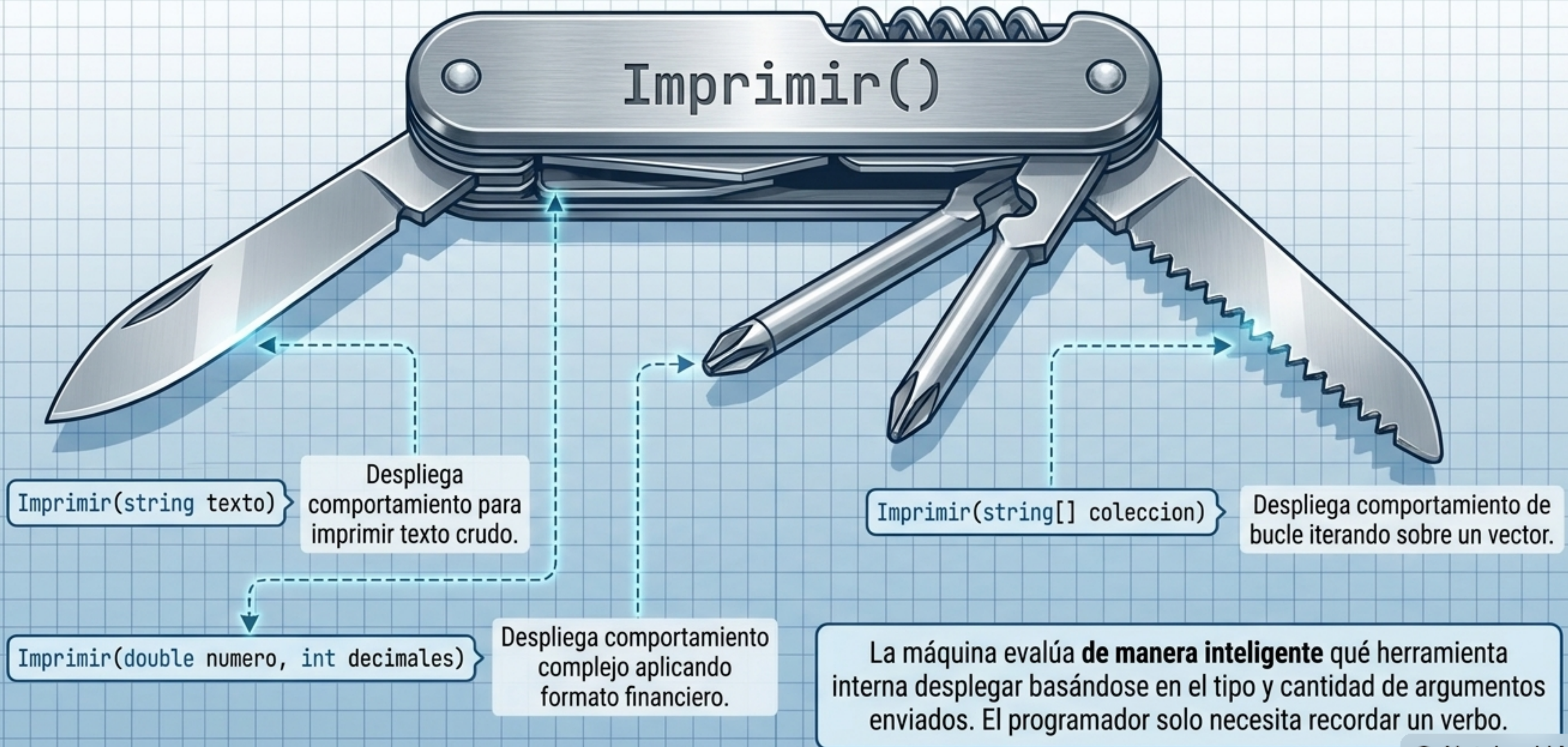
Relación Estricta. El objeto contenido depende totalmente del ciclo de vida del contenedor. Si el sistema muere, la parte muere.

```
public class Computadora
{
    public Procesador CPU { get; set; }
    public List<MemoriaRAM> Modulos { get; set; }
}
```



Relación Flexible (Tiene-un). La 'Computadora' tiene un 'Procesador'. Si se destruye la PC, el Procesador y la RAM aún existen y pueden moverse a otro sistema.

Polimorfismo Estático a Través de la Sobrecarga de Métodos



Redefiniendo las Matemáticas Básicas (Sobrecarga de Operadores)



```
public static Moneda operator +(Moneda m1, Moneda m2)
{
    if (m1.Divisa != m2.Divisa)
        throw new InvalidOperationException("Divisas distintas");
    return new Moneda(m1.Monto + m2.Monto, m1.Divisa);
}
```

Lógica de Negocio Blindada:
La sobrecarga permite usar sintaxis natural (+), pero incrusta reglas de seguridad para evitar sumar Bolivianos (BOB) con Dólares directamente.

SÍNTESIS: El Ecosistema P00 en Acción

SOBRECARGA:

Acciones que se adaptan dinámicamente al tipo de carga de entrada.

COMPOSICIÓN:

Subsistemas independientes
que se ensamblan
en uno mayor.

OBJETOS:

La construcción viva;
instancias físicas
en memoria.

ENCAPSULACIÓN:

Los guardias que protegen los datos internos (Propiedades).

CLASES:

CLASES: El plano conceptual que define la estructura del sistema.

- COMPOSICIÓN:

Subsistemas independientes
que se ensamblan en uno
mayor.

ESPECIFICACIONES TÉCNICAS Y REFERENCIAS DE ESTUDIO

DOCUMENTACIÓN OFICIAL

Microsoft Learn: C# Documentation
& OOP Fundamentals

El estándar de la industria para referencias
de sintaxis y arquitectura .NET.

ARQUITECTURA DE LENGUAJE

Skeet, Jon. 'C# in Depth'

Para comprender los engranajes internos
y la evolución del lenguaje.

PRÁCTICAS Y REFACTORIZACIÓN

Troelsen & Japikse. 'Pro C# 10'
Fowler, Martin. 'Refactoring'

Textos definitivos sobre la mejora continua
del diseño de clases.

REPOSITORIO



```
git clone https://github.com/OOP-Master-Class/ejemplos.git
```

Escanee para descargar el código fuente
y matrices de estudio.